

Sample Based Learning Methods —

(18)

Monte Carlo → It is used for broadly any estimation method that relies on repeated random sampling.

→ In the previous course, we talk about how RL is connected to DP. To use pure DP approach, the agent needs to know the environment transition probabilities.

* In some problems, we don't know these probabilities.

Monte Carlo methods for policy Evaluation →

$$V_{\pi}(s) \doteq E_{\pi} [G_t \mid s_t = s]$$

$$G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots + \gamma^n R_{n+1}$$

Note :- Monte Carlo methods can be used only for episodic tasks.

→ Since MC methods average over large number of returns ~~over~~ for each state.

These returns can be observed at the end of an episode!

MC prediction, for estimating $V = V_\pi \rightarrow$

- 1) Input: a policy π to be evaluated
- 2) Initialize:
 $V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$
 $\text{Returns}(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$
- 3) Loop forever (for each episode):
Generate an episode following π : $S_0, A_0, R_1, S_1, A_2, R_2, \dots, S_{T-1}, A_{T-1}, R_T$
 $G \leftarrow 0$
loop for each step of episode, $t = T-1, T-2, \dots, 0$:
 $G \leftarrow \gamma G + R_{t+1}$
Append G to $\text{Returns}(S_t)$
 $\left\{ V(S_t) \leftarrow \text{average}(\text{Returns}(S_t)) \right\}$
also ~~incremental~~ incremental update can be used here

Monte Carlo Methods for prediction :-

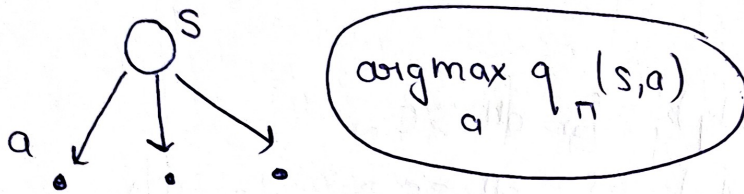
$$\text{New estimate} \leftarrow \text{oldest estimate} + \text{stepsize}(\text{target} - \text{oldest estimate})$$

Implications of Monte Carlo learning $\rightarrow$

- 1) We do not need to keep a large model of the environment.
- 2) We are estimating the value of an individual state independently of the values of other states.

Monte Carlo for Control →

- Here we collect the returns following a policy from state action pair and then take their average.
- action-values are useful for learning a policy.



- Here the agent must try all the actions in each state in order to learn their values. This is the problem of maintaining exploration in reinforcement learning. ← One way to do is ϵ -greedy! (other way is exploring starts)

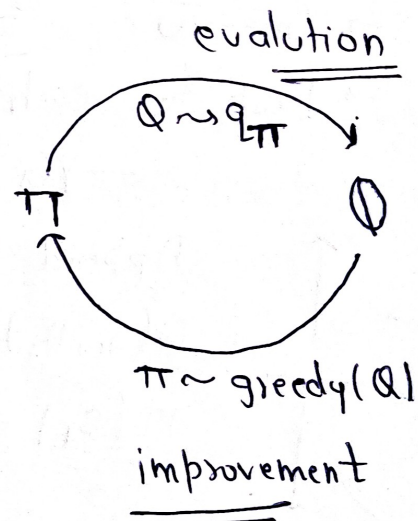
Generalized Policy Iteration (Using Monte Carlo) → (GPI)

1) Improvement:

$$\pi_{k+1}(s) = \arg\max_a q_{\pi_k}(s, a)$$

2) Evaluation:

Monte Carlo evaluation



✗ Monte Carlo control methods Combined policy improvement and policy evaluation on an episode by episode basis.

Monte Carlo ES (exploring starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in A(s)$ (arbitrarily), for all $s \in \mathcal{S}$

$Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in A(s)$

Returns $(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in A(s)$

Loop forever:

$\left\{ \begin{array}{l} \rightarrow \text{Choose } S_0 \in \mathcal{S}, A_0 \in A(S_0) \text{ randomly such that} \\ \text{all pairs have probability } > 0. \end{array} \right.$

$\rightarrow$ Generate an episode from S_0, A_0 , following π :
 $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$.

$\rightarrow G \leftarrow 0$

$\rightarrow$ Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Append G to Returns (S_t, A_t)

$Q(S_t, A_t) \leftarrow \text{average}(\text{Returns}(S_t, A_t))$

$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

exploring starts

Good for exploration

policy evaluation

policy improvement

Epsilon Soft policies →

(20)

→ Underlying exploring starts might limit the applicability of MC method. Therefore, ϵ -greedy exploration with MC is used to learn near optimal policies **

(Since, it is not always possible to start with random state.)

∅ ϵ -greedy policies are subset of a larger class of policies called epsilon soft policies. ϵ -soft policies take each action with probability at least $\frac{\epsilon}{|A|}$.

for e.g. { Uniform random policy is another notable ϵ -soft policy }

∅ ϵ -soft policies are always stochastic. ~~Deterministic~~

∅ If our policy always gives at least ϵ probability to each action, it is impossible to converge to a deterministic optimal policy. (However exploring starts can be used to find the optimal policy)!

→ Soft policies can only be used to find the optimal epsilon soft policy i.e. the policy with highest value in each state out of all the ϵ -soft policies.

? This policy perform worse than optimal policy?

MC Control (for ϵ -soft policies), estimates $\pi = \pi_*$

Algorithm parameters : small $\epsilon > 0$

Initialize :

$\pi \leftarrow$ an arbitrary soft policy.

{ same as previous algo }
:
}

Here exploring
starts step
won't be there

evolutn
=

improvement
=

$\uparrow Q(s_t, A_t) \leftarrow \text{average}(\text{Returns}(s_t, A_t))$

$\downarrow A^* \leftarrow \underset{a}{\operatorname{argmax}} Q(s_t, a)$ { with ties broken }
arbitrary

for all $a \in A(s_t)$:

$$\pi(a|s_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|A(s_t)|, & a = A^* \\ \epsilon/|A(s_t)|, & a \neq A^* \end{cases}$$

Note : { optimal ϵ -soft policy
is an ϵ -greedy policy }

$\Rightarrow$ Above, algorithm may not find optimal policy. It does
find the optimal ϵ -soft policy.

Off policy $\Rightarrow$

(21)

- ✗ In the exploration-exploitation tradeoff the key point that the agent must occasionally take suboptimal exploratory actions potentially receiving less reward.
- ✗ Off policy learning can deal with the exploration problem.
- ✗ ϵ -soft policies help with the continual exploration problem by having some probability of exploring on each time step. However the disadvantage is that these ϵ -soft policies are sub optimal _{in} both acting and learning.
- ✗ ϵ -soft policies are neither optimal policies for obtaining reward nor are the optimal for exploring to find best actions.

ON Policy

- 1) Improve and evaluate the policy being used to select actions.

OFF Policy

- 1) Improve and evaluate a different policy from the one used to select actions.
- 2) Here the agent learns about the policy (evaluation) from data generated by following a different policy.
i.e. the policy we are learning is off the policy that we are using for action selection.

- 3) for e.g. we could learn the optimal policy while following a totally random policy.
- 4) We call the policy that the agent is learning the target policy because it is the target of the agents. (denoted by π).
- 5) The value function that the agent is learning is based on the target policy.
- 6) e.g of target policy is optimal policy.
- 7) We call the policy that the agent is using to select actions, the behaviour policy because it defines our agents behaviour. (denoted by B)
- 8) The behavior policy is in charge of selecting actions for the agent.
- 9) Here we are decoupling both the policies because it provides another strategy for continual exploration.
- 10) If our agent behaves according to target policy it might only experience a small number of states.

(22)
11) If our agent can behave according to policy that favours exploration, it can experience much larger number of states.

12) One key rule of off policy learning is that the behaviour policy must cover the target policy.

i.e. if $\pi(a|s) > 0$ ★
then $b(a|s) > 0$ (must)

⇒ Here we can note that off policy learning is the strict generalization of on policy.

$(\pi(a|s) = b(a|s))$ i.e. target policy is equal to the behaviour policy

⇒ off policy learning allows learning an optimal policy from suboptimal behaviour.

Importance Sampling :-

- # It allow us to do off policy learning, learning with one policy while following another.
- # Use importance sampling to estimate expected value of a distribution using samples from a different distribution.

Derivation of importance Sampling :-

Sample : $x \sim b$

Random Variable

(~~sample~~ x that's being
sampled from probability
distribution b .)

Estimate : $E_{\pi}[x]$

(we ~~were~~ want to estimate the
expected value of x but with
respect to the target distribution
 π)

→ Because ' x ' is drawn from b , we cannot simply use the sample average to compute the expectation under π .

→ Sample average will give $E_b[x]$ instead !

$$E_{\pi}(x) = \sum_{x \in \mathcal{X}} x \cdot \pi(x)$$

$$E_{\pi}(x) = \sum_{x \in \mathcal{X}} x \cdot \pi(x) \frac{b(x)}{b(x)}$$

$$E_{\pi}(x) = \sum_{x \in \mathcal{X}} \left\{ \frac{\pi(x)}{b(x)} \right\} x \cdot b(x)$$

importance sampling trick! $\frac{\pi(x)}{b(x)} = p(x)$

$$E_{\pi}(x) = \sum_{x \in \mathcal{X}} x P(x) b(x) \quad (23)$$

consider this
as a new Random Variable

$$E_{\pi}(x) = \sum_{x \in \mathcal{X}} E_b[x P(x)]$$

$$\Rightarrow E_b[x P(x)] \approx \frac{1}{n} \sum_{i=1}^n x_i P(x_i), \quad x_i \sim b$$

→ off Policy Monte Carlo prediction →

$$V_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

≈ average (

Returns[0],

Returns[1],

Returns[2],)

← $a \sim b$

(i.e. these returns are
obtained by
following a
behaviour policy 'b')

≈ average (

P_0 Returns[0],

P_1 Returns[1],

P_2 Returns[2],)

Thus all we have
to do is to figure
out the value of
 P for each of the
sampled returns.

$$P = \frac{P(\text{trajectory under } \pi)}{P(\text{trajectory under } b)}$$

★★

⇒ This P corrects the distribution over entire trajectories, and so corrects the distribution over returns.

$$V_{\pi}(s) = E_b [PG_t | S_t = s]$$

$$\Rightarrow P(A_t, S_{t+1}, A_{t+1}, \dots, S_T | S_t, A_{t:T})$$

⇒ All the actions are sampled according to behaviour b .

$$= \underbrace{b(A_t | S_t) P(S_{t+1} | S_t, A_t)}_{\dots P(S_T | S_{T-1}, A_{T-1})} b(A_{t+1} | S_{t+1}) P(S_{t+2} | S_{t+1}, A_{t+1})$$

$\left\{ \begin{array}{l} P(A_t, S_{t+1} | S_t) \equiv \text{probability that the agent likes} \\ \text{action } A_t \text{ in state } S_t \text{ times the} \\ \text{probability that the environment} \\ \text{transitions into state } S_{t+1}. \end{array} \right\}$

$$\Rightarrow \prod_{k=1}^{T-1} b(A_k | S_k) P(S_{k+1} | S_k, A_k) = P(\text{trajectory under } b)$$

$$P_{t:T-1} \doteq \frac{P(\text{trajectory under } \pi)}{P(\text{trajectory under } b)}$$

$$\underbrace{P_{t:T-1}}_{\gamma P_0} \doteq \prod_{k=1}^{T-1} \frac{\pi(A_k | S_k) P(S_{k+1} | S_k, A_k)}{b(A_k | S_k) P(S_{k+1} | S_k, A_k)}$$

$$p_{t:T-1} \doteq \frac{\prod_{k=t}^{T-1} \pi(A_k | s_k)}{b(A_k | s_k)} \quad (24)$$

$$\Rightarrow E_b [p_{t:T-1} G_t | s_t = s] = V_\pi(s)$$

Every Visit MC prediction, for estimating $V \approx V_\pi$

Input : a policy π to be evaluated

Initialize :

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$\text{Return}(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0 \rightarrow$ Change #2 $\Rightarrow \{ G \leftarrow 0, w \leftarrow 1 \}$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$

$G \leftarrow \gamma G + R_{t+1}$

Append G to $\text{Returns}(s_t)$

$V(s_t) \leftarrow \text{average}(\text{Returns}(s_t))$

Change #1 $\Rightarrow \{ \text{Generate an episode following } b \}$

Change #3 $\Rightarrow \{ G \leftarrow \gamma w G + R_{t+1} \}$

Change #4 $\Rightarrow \left\{ w \leftarrow w \frac{\pi(A_t | s_t)}{b(A_t | s_t)} \right\}$

$\Rightarrow$ these changes are for off policy every visit MC, for estimating $V \approx V_\pi$

Note:- the episodes in off policy, is now generated following the behaviour policy. The return is corrected by new term w , which is the accumulated product of important sampling ratios on each time stamp of the episode.

$$\left\{ P_{t:T-1} = P_t P_{t+1} P_{t+2} \dots P_{T-2} P_{T-1} \right\}$$

←

-
- # This week was all about estimating value functions and finding optimal policies using only experience from the environment.
 - # MC method wait for the full return before updating their values. Therefore we use MC only for episodic MDPs.
 - # MC algorithms do not do sweeps over the state action ~~space~~ space like dynamic programming. So they need exploration mechanism to ensure that they learn about every state action pair.
 - # Exploration techniques →
 - 1) Exploring starts
 - 2) ϵ -soft policies
 - 3) off-policy

}

$\Rightarrow$ ON Policy

$\Rightarrow$ Can find optimal policy

}

$\Rightarrow$ Can find optimal ϵ -soft policy